protoST — guion de la charla (FAST, 15/10/2026)

Duración: núcleo de 30 minutos; módulos opcionales hasta 60. Las cifras de rendimiento salen de estos informes, todos de benchmarks sintéticos en una notebook (Ryzen 5 5500U):

- `benchmarks/reports/2026-10-12-protocore-2.14.6.md` (12/10): el binario publicado de protoST sobre protoCore 2.14.6 contra el mismo binario sobre 2.14.5, y CPython 3.14 en la misma máquina. De ahí salen el arranque, las comparaciones con CPython, los actores, la demo 2 y la memoria de K5;
- `benchmarks/reports/2026-10-12-adaptive-heap-default.md` (12/10): 0.6.2 contra 0.6.1, el controlador adaptativo del heap (actores, memoria, pausas);
- `benchmarks/reports/2026-10-11-actor-regression-bisection.md` (11/10): el ruido de la medición.

Si una cifra no está en uno de esos informes, no se dice.

Antes de empezar: terminal con fuente grande, `cd docs/talks/2026-10-15-fas/demos` en el repositorio (o, con el paquete instalado, `cd /usr/share/protoST/talks/2026-10-15-fas/demos`); `protost --version` debe decir `protoST 0.6.5`; la demo 4 necesita `python3`. Las grabaciones de respaldo están en `../recordings/` del repositorio (ver la checklist).

Núcleo (30 minutos)

0:00 — Quién y por qué (2 min)

- Esto es un experimento que busca entusiastas, no un producto que busca usuarios.
- protoST no es un Smalltalk mejor y no compite con el entorno vivo. Es otra cosa: poder leer y escribir el Smalltalk que conocen en lugares donde hoy Smalltalk no llega.
- Lo hace una persona con agentes de IA como herramientas; todo lo que muestro está en el repositorio y se puede verificar.

0:02 — Preguntas, no respuestas (3 min)

Planteadas como preguntas al público (no afirmarlas por ellos):

- ¿Cuánto les cuesta hoy poner una imagen como servicio detrás de otra cosa?
- ¿Cuántos núcleos usa su código Smalltalk cuando la máquina tiene doce?
- ¿Cómo integran lo que escriben en Smalltalk con el resto del stack (Python, JavaScript, colas, bases)?
- ¿Cómo comparten código como texto plano con quien no usa una imagen?

No decimos que protoST resuelve todo eso. Decimos que abre una forma distinta de intentarlo.

0:05 — Por qué ahora es posible: protoCore (5 min)

- protoCore: un núcleo de objetos en C++ sobre el que corren varios lenguajes (JavaScript, Python, Clojure, Scala, protoST).
- Tres propiedades, cada una con un ejemplo simple:
  - estructuras inmutables compartidas por estructura: una lista "modificada" es una nueva versión que comparte casi todo con la anterior;
  - hilos nativos sin lock global y un GC concurrente;
  - un objeto mutable es una referencia atómica a una instantánea inmutable: cada lectura ve una instantánea consistente.
- Desde Smalltalk los objetos siguen cambiando: `at:put:` en un `Array` o `add:` en una `OrderedCollection` modifican el objeto instalando una instantánea nueva. Los strings no: un String es un `ProtoString` de protoCore (una cuerda, un valor sin modificación en el lugar).
- Consecuencia práctica: pasar un objeto a otro hilo es pasar una referencia, sin copiarlo. El objeto queda compartido: lo que un lado cambia, el otro lo ve en su próxima lectura.

0:10 — DEMO 1: el código que ya conocen (5 min)

```
./run_demo.sh 01-familiar-code.st
```

Mostrar el archivo antes de correrlo (clases, `printOn:`, bloques, retorno no local en `Bank>>find:`, excepción propia con `retry`, `ZeroDivide` con `resume: 0`, fracciones exactas, `LargeInteger`). Puntos a remarcar:

- es un archivo de texto; una línea en blanco cierra un método;
- `new` envía `initialize`, `printOn:` gobierna cómo se imprime todo;
- la última línea imprime el total (270): un script muestra solo lo que imprime.

Respaldo: `scriptreplay` de `recordings/01-familiar-code`.

0:15 — Actores y paralelismo real (3 min)

- Cualquier objeto se vuelve actor con `asActor`; cada mensaje devuelve un `Future` enseguida (salvo los que preguntan por la referencia misma: identidad, igualdad, `hash`, `isNil`, `isActor`, `printString`, que el proxy responde en el acto); el actor procesa un mensaje por vez, así su estado son variables de instancia comunes, sin locks.
- Los actores comparten un pool de hilos nativos.
- `wait`, `thenDo:`, `whenAll:`. Un error dentro del actor vuelve con su clase al que espera.

0:18 — DEMO 2: el mismo cálculo en un hilo y en doce actores (3 min)

```
./run_demo.sh 02-actors-parallelism.st
```

Los dos resultados coinciden (4946 primos) y se muestran los tiempos medidos en ese momento; la última línea (`time speedup: ...x`) es la aceleración. Decir la que aparezca en pantalla, no una de memoria: depende de la máquina y de su carga. En la máquina de la charla, quieta y en modo AC, con protoCore 2.14.6: mediana 2,4× (2,1–2,6×), con un hilo en unos 715 ms y los doce actores en unos 295 ms. Remarcar: el código del cálculo no cambió; cambió quién lo ejecuta. Si preguntan por qué es menos que las 2,8× de antes: protoCore 2.14.6 hizo más rápido el código de un hilo (ver preguntas-dificiles.md).

Respaldo: `recordings/02-actors-parallelism`.

0:21 — Mundos que podría abrir, y DEMO 4 (6 min)

Ideas, no promesas:

- **Gemelos digitales**: un actor por componente físico; cada evento se procesa atómicamente; los componentes conversan por mensajes. (Si hay tiempo, DEMO 3.)
- **Servicios y conexión con otros sistemas**: un proceso que arranca en decenas de milisegundos, lee archivos y la entrada estándar, habla HTTP (cliente y servidor, también https) y TCP/UDP, y corre otros programas; sin imagen que desplegar. Los nombres son los de Pharo: `'datos.csv' asFileReference contents`, `Stdio stdin nextLine`, `Smalltalk arguments`.
- **Convivir con otros runtimes**: desde protoCore 2.6, un runtime embebido en un proceso puede leer y escribir los objetos de otro sin copiarlos. Lo probado es una dirección: protoScala lee y escribe objetos de protoST; la inversa (protoST usando objetos o módulos de otro runtime) todavía no funciona (`docs/INTEROP.md` §0). Los binarios instalados todavía no se cargan entre sí. Decirlo así.

DEMO 4: `./run_demo.sh 04-connected-twin.st`

Un programa en Python emite lecturas de sensores, un JSON por línea; el gemelo en protoST las lee por la entrada estándar (es un filtro Unix: `python3 04-connected-twin.feed | protost 04-connected-twin.st`), la bomba es un actor, publica su estado por HTTP mientras corre y registra cada lectura en un archivo. Mostrar el código: `Stdio stdin linesDo:`, `JSON parse:`, `HTTPServer on:handler:`, `writeStreamDo:`. Remarcar: otro lenguaje entrega los datos, protoST los procesa con actores y los expone por HTTP; nada de eso necesitó una imagen.

Respaldo: `recordings/04-connected-twin`.

0:27 — Lo que protoST no es (2 min)

Leer los titulares del capítulo 14 del tutorial: no hay imagen ni browser, una línea en blanco cierra un método, strings inmutables, símbolos cortos iguales a strings, metaclase delgada, `thisContext` no soportado, un actor que espera no atiende otros mensajes (y un ciclo de esperas se reporta como error). Mostrar que está todo escrito y verificado.

0:29 — Cómo participar (1 min)

- Repositorio, tutorial (el capítulo 14 es para ustedes), `docs/STATUS.md` con lo abierto.
- Lo que más ayuda: programas reales escritos como los escribirían ustedes, y reportes de todo lo que sorprenda.
- Preguntas.

Módulos (hasta 60 minutos)

M1 — protoCore por dentro (10 min)

Celdas de 64 bytes, enteros inline en el puntero, listas AVL y cuerdas (ropes) inmutables, `ProtoThread` y cómo protoST la usa para los workers. La instantánea atómica de un objeto mutable (dibujo en pizarra).

El recolector, tal como está en protoCore 2.14.6: concurrente; el barrido recorre varias cadenas a la vez y puede sumar hilos auxiliares; desde 2.14.6 la lista de trabajo del marcado tiene un tamaño fijo (8 MiB). Desde 0.6.2 protoST usa el controlador adaptativo del heap de protoCore (hasta 0.6.1, un techo fijo de 640 MB): más rápido en los programas de actores, con entre un quinto y la mitad de la memoria. El precio: el pico de memoria ya no está acotado (un bucle que asigna sin parar llegó a 0,88–3,5 GB); `PROTOCORE_HEAP_LIMIT_CELLS` lo fija. Para eso hubo que corregir algo propio: cada pausa stop-the-world recorría entero un arreglo de frames de 1 M de entradas por hilo (pausas de hasta 20–31 ms con seis a doce workers); desde 0.6.2 el arreglo crece bajo demanda y las pausas no pasan de 0,27 ms en los benchmarks. Con el techo fijo de 640 MB, un arreglo de 500 000 elementos llevaba el proceso a 1165 MB de RSS por la lista de trabajo del marcado; protoCore 2.14.6 la acota y el pico baja a 690 MB. Decirlo como medición (`KNOWN_ISSUES.md`, K5 y K6).

M2 — El gemelo digital paso a paso (8 min)

`./run_demo.sh 03-digital-twin.st`: la bomba como máquina de estados en un actor, tres sensores con 50 ms de E/S simulada leídos en paralelo en cada ciclo (el tiempo por ciclo en pantalla ronda los 50 ms, no 150), la alarma que detiene la bomba al pasar 75 grados. Recorrer el código.

Respaldo: `recordings/03-digital-twin`.

M3 — Rendimiento y método (6 min)

- Cada benchmark verifica su resultado; el harness falla si falta o no coincide (se mostró un bug que se veía como "mejora" en otro proyecto por no verificar).
- Benchmarks sintéticos en una notebook (Ryzen 5 5500U), informe del 12/10 sobre protoCore 2.14.6:
  - arranque 25 ms (CPython 32–35 ms);
  - frente a CPython, proceso completo con el mismo N: media geométrica 3,2–3,3×, dominada por el arranque;
  - solo el trabajo (arranque descontado): de unas 4–7× (bucle de enteros) a 30–35× (`fib`) y 55–64× (excepciones); la media geométrica salió entre 14× y 20× según la corrida, porque con N chicos la resta es sensible al ruido; decirlo como rango;
  - doce actores: 2,8× con el pool contra un worker; `saturation_big`, 2,5× con cuatro workers y 2,9× con seis, sin techo hasta seis;
  - protoCore 2.14.6 frente a 2.14.5: la demo 2 un 45 % más rápida en un hilo y un 35 % con doce actores; las excepciones 7 % más rápidas; el resto igual dentro del ruido;
  - 0.6.2 activó el controlador adaptativo del heap (informe del 12/10 sobre el heap: actores 16–43 % más rápidos, entre un quinto y la mitad de la memoria, y el pico ya no acotado); 0.6.1 corrigió la recolección en recursiones sin bucles (fib 29 se quedaba sin memoria).
- Decir dónde es más lento (despacho de métodos recursivo, excepciones) sin maquillarlo.

M4 — Hoja de ruta y preguntas abiertas (6 min)

Que los runtimes se carguen entre sí (el núcleo ya lo permite desde protoCore 2.6), `thisContext`, herramientas (el debugger DAP existe), rendimiento de envío de mensajes, WebSockets y HTTP/2. Preguntar al público qué les resultaría útil primero.